STIRE: sistema tutor inteligente con repetición espaciada para el aprendizaje de Fundamentos de Algoritmia

Jeider Jair Gómez Oviedo[1], Pedro Gabriel Romero Mendivil[1], José López[1], Julio Galvis[1], Jorge Cervantes[1]
[1] Licenciatura en Informática, Facultad de Educación y Ciencias Humanas, Universidad de Córdoba, Montería, Colombia.

RESUMEN

El aprendizaje de la programación exige retener y aplicar conceptos que se acumulan de una semana a otra, y en grupos numerosos el docente no alcanza a acompañar a cada estudiante. Este artículo presenta el diseño, el desarrollo y una evaluación preliminar de STIRE, un sistema tutor inteligente web para el curso Fundamentos de Algoritmia de la Universidad de Córdoba (Colombia). STIRE combina ejercicios calificados automáticamente, un tutor conversacional basado en un modelo de lenguaje que orienta con preguntas y pistas, y un calendario de repasos que adapta el algoritmo SM-2 al ritmo de un semestre. El desarrollo siguió la metodología MODESEC en iteraciones semanales. La evaluación preliminar la realizó el equipo de desarrollo con dos listas de chequeo, de interfaz gráfica y de sistemas tutores inteligentes, y con la escala de usabilidad SUS. Tras las mejoras, el cumplimiento llegó a 100 % y 95,8 %, respectivamente, y la usabilidad obtuvo 81,5 puntos. Una evaluación previa hecha con inteligencia artificial resultó más generosa que la revisión humana en 9 de 28 criterios. El uso del tutor fue bajo, en parte porque cada estudiante debía configurar su propia clave de acceso al modelo.

PALABRAS CLAVES: aprendizaje de la programación; inteligencia artificial; repetición espaciada; sistemas tutores inteligentes; usabilidad.

ABSTRACT

Learning to program requires retaining and applying concepts that build on each other week after week, and in large groups instructors cannot support every student. This paper presents the design, development, and a preliminary evaluation of STIRE, a web-based intelligent tutoring system for the Fundamentals of Algorithms course at the University of Córdoba (Colombia). STIRE combines automatically graded exercises, a conversational tutor based on a language model that guides students with questions and hints, and a review schedule that adapts the SM-2 algorithm to the pace of an academic term. Development followed the MODESEC methodology in weekly iterations. The development team carried out the preliminary evaluation with two checklists, one for the graphical interface and one for intelligent tutoring systems, and with the System Usability Scale. After the improvements, compliance reached 100% and 95.8%, respectively, and usability scored 81.5. An earlier AI-based evaluation was more lenient than the human review in 9 of 28 criteria. Tutor use was low, partly because each student had to set up their own access key to the model.

KEY WORDS: artificial intelligence; intelligent tutoring systems; programming learning; spaced repetition; usability.

I. INTRODUCCIÓN

Aprender a programar exige retener y aplicar conceptos que se apoyan unos en otros: una variable mal entendida dificulta los condicionales, y estos, los ciclos. En los cursos introductorios, además, el docente rara vez puede acompañar a cada estudiante en el momento en que se equivoca.

La investigación ofrece varias respuestas. Los sistemas tutores inteligentes que guían la solución paso a paso alcanzan efectos cercanos a los de la tutoría humana (VanLehn, 2011). Repartir la práctica en el tiempo retiene mejor que concentrarla, al menos en tareas de memoria verbal (Cepeda et al., 2006), y resolver una prueba fortalece el recuerdo más que volver a estudiar el material (Roediger y Karpicke, 2006). La retroalimentación, por su parte, ayuda cuando le dice al estudiante hacia dónde va, cómo va y qué sigue (Hattie y Timperley, 2007).

La inteligencia artificial generativa agregó otra posibilidad: un asistente conversacional que acompaña

ISSN: 2619–2659 https://doi.org/10.21897/26192659.3902

al estudiante en cualquier momento. En el curso CS50 de la Universidad de Harvard, por ejemplo, se incorporó un asistente diseñado para guiar a los estudiantes sin entregarles las soluciones (Liu et al., 2024).

Este trabajo reúne esas ideas en un solo sistema para un curso concreto. Su objetivo fue **desarrollar y evaluar de forma preliminar STIRE, un sistema tutor inteligente web con repetición espaciada para el curso Fundamentos de Algoritmia de la Licenciatura en Informática de la Universidad de Córdoba.**

II. METODOLOGÍA Y DESARROLLO

A. Enfoque y contexto

Se trata de una investigación aplicada de desarrollo tecnológico. El software se construyó con MODESEC, una metodología que organiza el diseño de software educativo a partir de las competencias que se quieren desarrollar (Caro Piñeres et al., 2009), en iteraciones semanales, por un equipo de cinco estudiantes de la Licenciatura en Informática de la Universidad de Córdoba (Montería, Colombia), entre agosto y octubre de 2026. En el sistema se cargaron dos cursos: Fundamentos de Algoritmia y Pensamiento algorítmico.

B. Principios de diseño

Practicar resolviendo. Cada lección se trabaja con ejercicios de siete tipos, entre ellos programas en JavaScript que se ejecutan contra casos de prueba, y cada repaso es un ejercicio y no una relectura (Roediger y Karpicke, 2006). **Retroalimentación al entregar.** El estudiante ve el resultado de cada caso de prueba al momento (Hattie y Timperley, 2007). **Ayuda graduada.** El tutor empieza con una pista y sube a preguntas guía si el estudiante sigue fallando; no escribe la solución. **Un perfil descriptivo.** El perfil de aprendizaje le muestra al estudiante sus errores frecuentes y sus hábitos de estudio, pero no cambia el contenido que ve. **Funciones del docente opcionales.** El docente puede publicar contenido, revisar entregas o llevar notas, pero el sistema funciona sin que lo haga.

C. Arquitectura

STIRE separa el cliente web del servidor (Figura 1). El cliente usa Nuxt 3, Vue 3 y Pinia, y se auditó con axe-core frente a las pautas WCAG 2.2 (World Wide Web Consortium [W3C], 2023): los 196 elementos señalados en 78 vistas de computador y celular se corrigieron. El servidor usa NestJS y TypeORM sobre MariaDB y corre en contenedores Docker. El tutor consulta el modelo Gemini de Google con la clave propia de cada estudiante.



Figura 1. Arquitectura de STIRE y módulos del servidor.

D. Repetición espaciada

Los repasos adaptan el algoritmo SM-2 (Woźniak, 1990). En el original, el primer repaso llega al día siguiente, el segundo a los seis días y, desde el tercero, el intervalo anterior se multiplica por un factor de facilidad que empieza en 2,5, cambia según la calidad de la respuesta y nunca baja de 1,3; un fallo reinicia el ciclo. STIRE hace tres cambios. El segundo repaso llega a los tres días, para que los dos primeros caigan en la misma semana de clase. La calidad sale del resultado del ejercicio, sin preguntarle nada al estudiante: falla (1), acierta tras varios intentos (3), acierta al primer intento (4) o acierta al primer intento después de declararse seguro (5). Y ningún intervalo supera 60 días. La Tabla 1 muestra los intervalos que resultan.

ISSN: 2619–2659 https://doi.org/10.21897/26192659.3902

Tabla 1. Días hasta cada repaso según el resultado del estudiante

Resultado en cada repaso	Días hasta los repasos 1 a 6
Acierta al primer intento	1 · 3 · 8 · 20 · 50 · 60
Acierta tras varios intentos	1 · 3 · 6 · 12 · 22 · 37
Falla	vuelve a 1 día

E. Evaluación preliminar

Se usaron dos listas de chequeo: una de interfaz gráfica, basada en Pressman y Maxim (2019), con 16 criterios, y otra de sistemas tutores inteligentes, basada en Caro Piñeres (2015), con 12. Cada criterio vale 2 puntos si se cumple, 1 si se cumple parcialmente y 0 si no se cumple, de modo que las listas suman 32 y 24 puntos. Se aplicaron en cuatro momentos: el 4 de octubre, por el equipo con apoyo de un asistente de inteligencia artificial; el 6 de octubre, sobre la versión 2.0, solo por la inteligencia artificial; entre el 9 y el 17 de octubre, sobre la misma versión, por cada integrante como usuario, y el 17 de octubre, después de las correcciones.

La usabilidad se midió con la escala SUS (Brooke, 1996), interpretada con los rangos de Bangor et al. (2008). Entre el 5 y el 16 de octubre, el equipo usó el sistema durante dos semanas con los dos cursos, en un diseño cruzado: cada integrante fue docente o estudiante de un curso y cambió de curso en la segunda semana. El uso del tutor se tomó de los registros del servidor.

F. Uso de inteligencia artificial

En el desarrollo se usaron asistentes de programación con inteligencia artificial (Claude Code, Codex y Google Antigravity) para escribir y revisar código y documentación; cada cambio pasó por pruebas automáticas y por la revisión del equipo. En la preparación de este artículo se usó un asistente para estructurar el texto y verificar las referencias. Los autores revisaron el contenido y responden por él (Giraldo Cardozo y Muñoz Vargas, 2025).

III. RESULTADOS Y DISCUSIÓN

A. El sistema

La Figura 2 muestra una respuesta real del tutor. Ante la pregunta «No sé cómo sacar el perímetro», el tutor recuerda la fórmula del enunciado y le pide al estudiante escribir la operación con sus propias variables, en lugar de escribirla por él. La Figura 3 muestra la experiencia del estudiante en el celular: el inicio le dice cuál es su siguiente paso y cuánto domina la lección (a), y el ejercicio presenta el enunciado, un ejemplo y el editor, con botones para probar el código sin gastar intentos y para entregarlo (b).

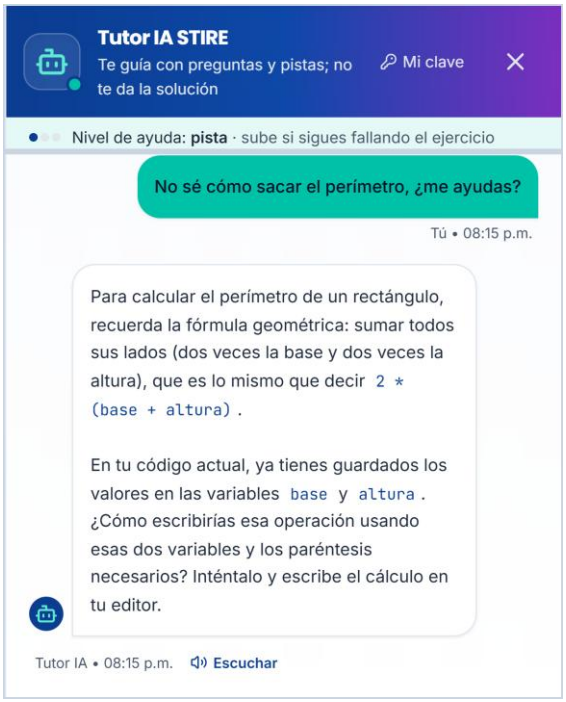


Figura 2. Respuesta del tutor: una pista y una pregunta, no la solución.

B. Listas de chequeo: inteligencia artificial frente a revisión humana

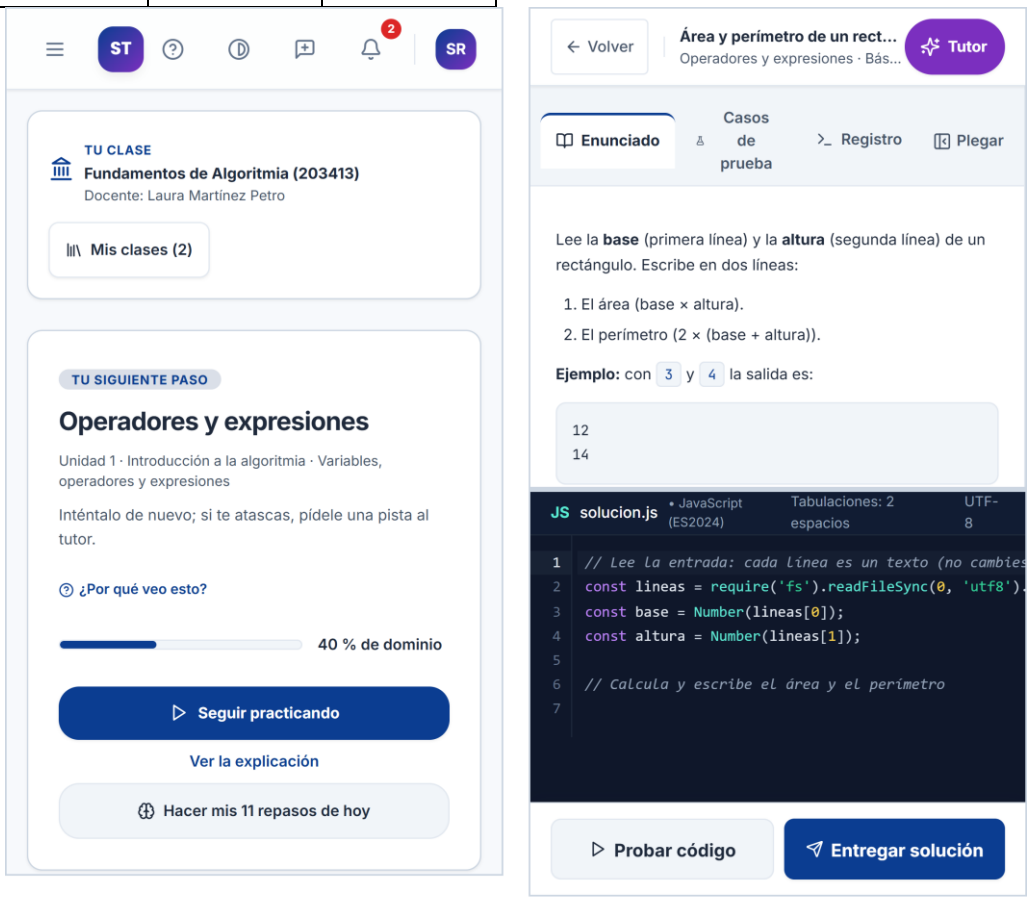
La Tabla 2 resume los cuatro momentos. La evaluación hecha solo por la inteligencia artificial fue más generosa que la revisión humana en 9 de los 28 criterios y más estricta en uno. Acertó en lo que se puede comprobar en el código y falló en lo que solo nota quien usa la aplicación: por ejemplo, dio por cumplido que el estudiante supiera cuánto dominio le faltaba para abrir el siguiente módulo, y un integrante, al usarla, no lo encontró. Por eso se tomó como oficial la revisión humana.

ISSN: 2619–2659 https://doi.org/10.21897/26192659.3902

Tabla 2. Puntos de las listas de chequeo en cada momento

Momento	Interfaz (32 pts.)	Tutores (24 pts.)
4 oct.: equipo con apoyo de IA	20 (62,5 %)	14 (58,3 %)
6 oct.: solo IA, versión 2.0	31 (96,9 %)	24 (100 %)
9–17 oct.: equipo como usuario, versión 2.0	27 (84,4 %)	20 (83,3 %)
17 oct.: tras las correcciones	32 (100 %)	23 (95,8 %)

La subida a 100 % y 95,8 % debe leerse con cautela: quienes evaluaron son los mismos que desarrollaron, y los criterios orientaron las mejoras, así que un aumento era esperable.



(a) **(b)**

Figura 3. STIRE en el celular: (a) inicio con el siguiente paso; (b) ejercicio de programación.

C. Usabilidad y uso del tutor

Cinco integrantes respondieron la escala SUS dentro del sistema, con un promedio de 81,5, dentro del rango aceptable según Bangor et al. (2008). Es una medida de referencia, porque la dieron los propios desarrolladores.

El tutor, en cambio, casi no se usó. El 12 de octubre, solo 4 de las 17 cuentas de estudiante tenían guardada su clave de acceso al modelo, no hubo mensajes al tutor en las 24 horas anteriores y las respuestas medidas tardaban entre 3 y 29 segundos. La clave personal parece ser la principal barrera: en despliegues como el de CS50, el asistente lo provee el curso (Liu et al., 2024).

D. Discusión

STIRE aplica en un mismo sistema la práctica distribuida, la práctica de recuperación y la ayuda graduada. Sin embargo, la evidencia que respalda cada idea proviene de otros contextos: los tutores que revisó

ISSN: 2619–2659 https://doi.org/10.21897/26192659.3902

VanLehn (2011) no usaban modelos de lenguaje, y los estudios de Cepeda et al. (2006) son de memoria verbal, no de programación. Además, SM-2 programa los repasos con reglas fijas, mientras que los modelos que aprenden de los datos de cada estudiante pueden programarlos mejor (Tabibian et al., 2019). Lo que muestra esta evaluación es que el sistema funciona y que su interfaz mejoró; su efecto en el aprendizaje está por medir.

IV. CONCLUSIONES

STIRE reúne en un sistema web el acompañamiento de un tutor, la retroalimentación inmediata y los repasos espaciados, adaptados al ritmo de un semestre. La evaluación preliminar mostró una mejora clara de la interfaz y de las funciones de tutoría, y dejó una lección metodológica: la evaluación con inteligencia artificial sirve para revisar lo verificable, pero no reemplaza la prueba con usuarios. Las principales limitaciones son que evaluaron los propios desarrolladores y que no se ha medido el aprendizaje. Como trabajo futuro se proponen una prueba con estudiantes del curso, una clave de acceso al modelo provista por la institución, el repaso intercalado de varias lecciones y un programador de repasos que aprenda de los datos.

V. REFERENCIAS

Bangor, A., Kortum, P. T., y Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human–Computer Interaction, 24*(6), 574–594. https://doi.org/10.1080/10447310802205776

Brooke, J. (1996). SUS: A «quick and dirty» usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester y I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

Caro Piñeres, M. F. (2015). *Metamodel for personalized adaptation of pedagogical strategies using metacognition in intelligent tutoring systems* [Tesis doctoral, Universidad Nacional de Colombia]. https://repositorio.unal.edu.co/handle/unal/55505

Caro Piñeres, M. F., Toscano Miranda, R. E., Hernández Rozo, F. M., y David Lobo, M. E. (2009). Diseño de software educativo basado en competencias. *Ciencia e Ingeniería Neogranadina, 19*(1), 71–98. https://doi.org/10.18359/rcin.311

Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., y Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin, 132*(3), 354–380. https://doi.org/10.1037/0033-2909.132.3.354

Giraldo Cardozo, J. C., y Muñoz Vargas, I. C. (2025). Comunicación científica y ética en la era de la inteligencia artificial. *Acta Scientiæ Informaticæ, 9*(2). https://doi.org/10.21897/26192659.4189

Hattie, J., y Timperley, H. (2007). The power of feedback. *Review of Educational Research, 77*(1), 81–112. https://doi.org/10.3102/003465430298487

Liu, R., Zenke, C., Liu, C., Holmes, A., Thornton, P., y Malan, D. J. (2024). Teaching CS50 with AI: Leveraging generative artificial intelligence in computer science education. En *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (pp. 750–756). ACM. https://doi.org/10.1145/3626252.3630938

Pressman, R. S., y Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Roediger, H. L., y Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science, 17*(3), 249–255. https://doi.org/10.1111/j.1467-9280.2006.01693.x

Tabibian, B., Upadhyay, U., De, A., Zarezade, A., Schölkopf, B., y Gomez-Rodriguez, M. (2019). Enhancing human learning via spaced repetition optimization. *Proceedings of the National Academy of Sciences, 116*(10), 3988–3993. https://doi.org/10.1073/pnas.1815156116

VanLehn, K. (2011). The relative effectiveness of human tutoring, intelligent tutoring systems,

and other tutoring systems. *Educational Psychologist, 46*(4), 197–221. https://doi.org/10.1080/00461520.2011.611369

Woźniak, P. A. (1990). *Optimization of learning* [Tesis de maestría, Universidad Tecnológica de Poznań]. https://www-v1.supermemo.com/archives1990-2015/english/ol/sm2

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/